

端侧 AI / 大文件模型加载 / 连续文件访问需求增长

核心判断

先不要把全部问题都笼统称为“粒度失配”。
第一层是页缓存对象过小导致的大块 I/O 利用不足。

问题层 1: 4KB 页缓存粒度过小

页缓存对象数量过多, 分配、加锁、映射查询与状态维护被拆成大量 4KB 单元, 难以充分利用底层块层和闪存设备已经具备的大块连续 I/O 能力。

第一步技术: Large Folio

扩大页缓存对象粒度
4KB page → 16KB / 64KB folio

回应第一层问题

进一步暴露

问题层 2: Large Folio 引入后, 跨层边界不再天然对齐

一个更大的 folio 会同时覆盖多个文件系统块、多个子区间和多个 bio, 导致映射边界、提交边界以及 I/O 完成生命周期之间出现新的不一致。

第二步技术: iomap

逐块映射 → 字节区间映射
统一 buffered I/O 与 writeback 的 I/O 组织方式

回应第二层问题

继续深入

问题层 3: F2FS 私有机制与 iomap 通用框架发生兼容冲突

F2FS 的 GC/原子写/压缩等私有语义原本直接编码在页缓存对象中, 而 iomap 需要同一对象承载 per-folio 状态与完成计数, 因而必须进行专属扩展。

第三步技术: F2FS 专属扩展

统一状态结构
GC / compression / private 语义协同适配

回应第三层问题

本文目标: 在 F2FS 上完成 Large Folio + iomap 的全路径 buffered I/O 适配, 使页缓存对象粒度、区间映射语义与 F2FS 私有机制在普通读写、写回、GC 与压缩文件路径中重新对齐。

关键变化

Large Folio 改变的是缓存对象边界, 并不会自动改变 F2FS 块级语义与回调组织方式。

F2FS 特有复杂性

GC、compression、atomic write 等路径会放大兼容与生命周期问题。